Azure Landing Zone with Bicep

Implementation Solution Guide

Version 1.0.0 · 2026-07-09

Contents

Executive Summary — Azure Landing Zone with Bicep 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Azure Landing Zone with Bicep 5

What an Azure landing zone is . . . 5

How Bicep works . . . 5

What the landing zone contains . . . 5

What you can control . . . 6

Limitations (be honest with buyers) . . . 6

Authentication and connections used . . . 6

The standard deployment model . . . 6

2. Requirements — Azure Landing Zone with Bicep 7

R1. Tenant-root (or intermediate) management-group access . . . 7

R2. Bicep CLI at a supported version . . . 7

R3. Deployment-stacks availability in the target environment . . . 7

R4. Resource providers registered . . . 7

R5. Address-space plan for hub-spoke (non-overlapping) . . . 8

R6. Naming and tagging convention encoded as parameters . . . 8

R7. Pipeline approval gate and reviewer group . . . 8

Global Requirements — DevOps (category-level) 8

G1. Target cloud subscription/tenant with sufficient rights . . . 9

G2. IaC tooling installed and versioned . . . 9

G3. Source control repository . . . 9

G4. CI/CD platform and service connection . . . 9

G5. Remote state / deployment backend . . . 9

G6. Secrets management . . . 9

G7. Naming, tagging, and region standards . . . 10

G8. Change-control and non-production target . . . 10

3. Implementation — Azure Landing Zone with Bicep 10

Naming you will create (reuse these exact names) . . . 10

Step 1 — Assess: inventory the current estate (read-only) . . . 11

Step 2 — Configure: scaffold the Bicep repo . . . 11

Step 3 — Configure: deploy the management-group hierarchy (non-prod first) . . . 11

Step 4 — Configure: assign policy guardrails in AUDIT . . . 12
Step 5 — Configure: apply the identity / RBAC baseline . . . 12
Step 6 — Configure: deploy hub-spoke networking and DNS (non-prod) . . . 12
Step 7 — Pilot: validate the non-prod landing zone . . . 12
Step 8 — Wire the CI/CD pipeline (what-if → approval → deploy) . . . 13
Step 9 — Expand: deploy to production, then promote guardrails to DENY . . . 13
5. Testing — Azure Landing Zone with Bicep 13
Test 1 — Management-group hierarchy exists . . . 13
Test 2 — Bicep builds and what-if is clean . . . 14
Test 3 — Guardrails assigned in AUDIT (not deny yet) . . . 14
Test 4 — Audit compliance picture is understood . . . 14
Test 5 — Least-privilege RBAC baseline . . . 14
Test 6 — Hub-spoke connectivity and DNS . . . 14
Test 7 — Deployment stack tracks and can roll back . . . 15
Test 8 — Pipeline enforces what-if and approval . . . 15
Exit criteria . . . 15
6. Troubleshooting — Azure Landing Zone with Bicep 15
Deployment authorization / scope . . . 15
Bicep build / what-if . . . 16
Deployment stacks / rollback . . . 16
Policy . . . 16
RBAC / identity . . . 16
Networking . . . 17
Pipeline . . . 17
7. Best Practices — Azure Landing Zone with Bicep 17
Keep it clean . . . 17
Governance and decisions that belong to the customer . . . 18
Where to go next . . . 18
8. Deliverables Summary — Azure Landing Zone with Bicep 18
Documentation . . . 18
IaC and automation artifacts . . . 19
What the buyer receives . . . 19
Outcome . . . 20
Appendix A — PowerShell Scripts 21

Executive Summary — Azure Landing Zone with Bicep

White-label · DevOps

Most Azure estates start as a single subscription clicked together in the portal: no management-group hierarchy, no policy guardrails, over-privileged owners, ad-hoc networking, and nothing in source control. It works until it doesn't — the estate drifts, no two subscriptions look alike, and there is no repeatable way to stand up the next one. This solution builds an **Azure landing zone as code** using **Bicep** — Microsoft's native infrastructure-as-code language — so the foundation is governed, versioned, and repeatable without leaving the Microsoft toolchain.

This is a complete, **deploy-it-today** package for standing up a Bicep landing zone, written for an administrator who has used the Azure portal but has never authored Bicep, assigned Azure Policy as code, or run an IaC pipeline.

What you get

- A plain-language **Solution Overview** of the landing-zone pattern — management groups, Azure Policy and initiatives as Bicep, an identity/RBAC baseline, deployment stacks, and hub-spoke networking with DNS.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (the category-wide ones live in the separate Global Requirements).
- An **Implementation Guide** of precise, ordered steps — scaffold the repo, deploy the management-group hierarchy, assign policy in **audit**, apply the RBAC baseline, deploy hub-spoke networking, then wire the CI/CD pipeline with **what-if** and approvals.
- Five IaC / automation artifacts: a read-only **PowerShell discovery** script, the core `main.bicep` landing-zone template, a **PowerShell what-if validation** script, an `azure-pipelines.yml` pipeline (what-if → approval → deploy), and a **PowerShell posture report**.
- **Testing** and **Troubleshooting** guides for a confident, pilot-first rollout.

What this delivers

A **management-group hierarchy** deployed as Bicep, **Azure Policy initiatives** assigned in audit and promoted to deny on a controlled gate, an **identity/RBAC baseline** with custom roles and no standing Owner, **hub-spoke networking** with a central firewall subnet and private DNS, all deployed through **Azure deployment stacks** and a **CI/CD pipeline** that runs **what-if first** and requires approval before every production deploy — proven against a non-production scope before it touches production.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — every apply is preview-first (`what-if`), reviewed in a pull request, and approval-gated, with a documented rollback via deployment-stack delete/update.

Outcome

A Microsoft-native, versioned Azure foundation expressed entirely as Bicep in Git — management groups, policy guardrails, least-privilege identity, and hub-spoke connectivity — governed and repeatable, deployable by a single administrator and validated before you scale to the rest of the estate.

1. Solution Overview — Azure Landing Zone with Bicep

What an Azure landing zone is

An **Azure landing zone** is a pre-built, governed foundation you deploy *before* you put workloads in Azure. Instead of one flat subscription, you get a **management-group hierarchy** (a tree that lets one control apply to many subscriptions), **policy guardrails** that keep every subscription compliant, an **identity and RBAC baseline** so access is least-privilege from day one, and a **hub-spoke network** that gives workloads a consistent, centrally-controlled connectivity model. This solution builds that landing zone as **Bicep** — Microsoft's native, declarative infrastructure-as-code language that compiles to Azure Resource Manager (ARM) — so the entire foundation is code in Git, not clicks in a portal.

How Bicep works

Bicep is a domain-specific language for describing Azure resources. You write `.bicep` files that declare *what* you want (a management group, a policy assignment, a virtual network); the Bicep CLI transpiles them to ARM JSON; and Azure deploys them idempotently — running the same file twice converges to the same state rather than creating duplicates. Two features make it safe for a landing zone:

- `what-if` — before any deployment, `az deployment ... what-if` shows exactly which resources will be **created, modified, or deleted**. Nothing is applied until you have read the preview.
- **Deployment stacks** — a deployment stack is a managed grouping of resources deployed from Bicep. It tracks everything it created, can **deny** out-of-band changes to those resources (drift protection), and can be **updated or deleted as a unit** — which is your rollback.

Because Bicep targets ARM directly, there is **no third-party state file and no separate provider to manage** — the deployed Azure resources are the source of truth, and Azure itself stores deployment history.

What the landing zone contains

1. **Management-group hierarchy** — a root intermediate group with child groups (for example `Platform`, `Landing Zones`, `Sandbox`, `Decommissioned`) so one policy or role assignment covers many subscriptions from a single point.
2. **Azure Policy and initiatives as Bicep** — policy definitions and initiative (policy set) *assignments* declared in `.bicep` and assigned at management-group scope, in **audit** effect first and promoted to **deny** on a controlled gate.
3. **Identity / RBAC baseline** — custom role definitions and least-privilege role assignments as Bicep, removing standing Owner in favor of scoped, purpose-built access (and PIM-eligible assignments where Entra ID P2 is licensed — *verify current availability in your tenant*).
4. **Hub-spoke networking and DNS** — a central **hub** virtual network (gateway subnet, a firewall/`AzureFirewallSubnet`, shared services) and **spoke** VNets peered to it, plus **private DNS**

zones for private-endpoint name resolution.

5. **Deployment stacks + CI/CD** — the whole thing deployed via deployment stacks through an **Azure DevOps or GitHub Actions** pipeline that runs `what-if`, requires approval, then deploys.

What you can control

• **Hierarchy shape** — which management groups exist and which subscriptions sit under each.

• **Guardrail effect** — audit (report) or deny (block) per policy/initiative, moved on a gate.

• **Least-privilege access** — custom roles, removal of standing Owner, scope-narrowed assignments.

• **Network topology** — hub address space, spoke count and ranges, peering, firewall subnet, private DNS zones.

• **Rollout scope** — non-production management group first, production only after validation and approval.

Limitations (be honest with buyers)

• A landing zone governs *how* resources are created; it does not migrate or re-home resources that **already exist** outside it — those are brought under management separately.

• **Deployment stacks** and some policy effects/features may be in preview or vary by region and by Azure environment (Commercial vs. Government) — *verify current availability in your tenant before relying on them.* The specific ARM API versions in `main.bicep` are marked to verify.

• Certain capabilities require licensing — for example **PIM** for eligible role assignments needs **Entra ID P2** — *verify current availability in your tenant.*

• Moving guardrails to **deny** can block legitimate deployments; this is exactly why the model pilots in audit against a non-production scope first and keeps a rollback.

• This is the platform foundation; it does not replace workload security, backup, or patching.

Authentication and connections used

• **Azure CLI / Az PowerShell** signed in as an identity with **Owner** (or Owner + User Access Administrator) at the deployment scope, for local `what-if` and discovery.

• A **CI/CD service connection** to Azure using **workload identity federation (OIDC)** — no stored secret — scoped to the deployment target (per Global Requirements G4).

• **Microsoft Graph / Entra** is touched only for role and PIM assignment; the network and policy work is pure ARM.

The standard deployment model

Assess the current estate (subscriptions, existing policy/RBAC/network) → configure the Bicep repo, hierarchy, policy, identity, and network modules → pilot by deploying to a non-production management group with policy in audit → validate with what-if, the testing script, and a policy-compliance pass → expand to production behind pipeline approval, then promote guardrails to deny. *The repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****,*

what-if-first and approval-gated, with deployment-stack rollback.

2. Requirements — Azure Landing Zone with Bicep

The category-wide prerequisites (target subscription/tenant with rights, IaC tooling, source control, CI/CD platform and service connection, remote state / deployment backend, secrets management, naming/tagging/region standards, and a non-production target) are covered once in **Global Requirements — DevOps** and are **not repeated here**. Confirm those first. Below are the prerequisites **specific to a Bicep landing zone**. Each item: **Description / Why required / How to verify / How to implement if missing.**

R1. Tenant-root (or intermediate) management-group access

Description: The identity deploying the hierarchy can create and assign at the management-group scope — ideally an **intermediate** management group under the tenant root, not the tenant root itself. **Why required:** The landing zone is deployed *at* management-group scope; without rights there, the hierarchy, policy, and role assignments cannot be created. **How to verify:** In the portal go to **Management groups** and confirm you can see the hierarchy; run `az account management-group list`. Confirm your identity has **Owner** or **Contributor + User Access Administrator** at the target management group. **How to implement if missing:** A Global Administrator must first enable management groups (**Management groups** → **Start using management groups**) and grant your identity the required role at the intermediate group. *Elevating access at tenant root is the customer's governance decision.*

R2. Bicep CLI at a supported version

Description: The Bicep CLI (bundled with a current Azure CLI, or installed standalone) at a version that supports deployment stacks and the `.bicep` features used here. **Why required:** Older Bicep builds transpile differently and may not support `az stack` / deployment-stack commands. **How to verify:** Run `az bicep version` and `az version`. *Verify current supported versions — Bicep and the Azure CLI update frequently.* **How to implement if missing:** Run `az bicep upgrade` (or install the standalone Bicep CLI) and pin the version in the repo (`bicepconfig.json` / a tool-version file).

R3. Deployment-stacks availability in the target environment

Description: Azure deployment stacks available in the subscription/management group and Azure environment (Commercial, or a Government/sovereign cloud) you are targeting. **Why required:** The solution deploys and rolls back through deployment stacks; if the feature or its API version is not available, you fall back to standard `az deployment mg/sub create`. **How to verify:** Run `az stack mg list` (or `az stack sub list`) against the target scope; if the command errors as unrecognized, upgrade the CLI. *Verify deployment-stacks availability and the exact API version in your tenant/cloud — some features are in preview or region-limited.* **How to implement if missing:** Upgrade the Azure CLI; if stacks are unavailable in your cloud, use the provided pipeline's standard-deployment fallback path (documented in the Implementation Guide).

R4. Resource providers registered

Description: The ARM resource providers the landing zone uses — at minimum `Microsoft.Authorization` (policy/RBAC), `Microsoft.Management` (management groups), `Microsoft.Network` (VNets, firewall, private DNS), and `Microsoft.Resources`. **Why required:** A deployment against an unregistered provider fails with a `MissingSubscriptionRegistration`-style error. **How to verify:** `az provider list --query "[?registrationState=='Registered'].namespace"` on each target subscription. **How to implement if missing:** `az provider register --namespace Microsoft.Network` (repeat per provider); registration is asynchronous — re-check until `Registered`.

R5. Address-space plan for hub-spoke (non-overlapping)

Description: A reserved, **non-overlapping** IP address plan — a CIDR block for the hub VNet and distinct blocks per spoke — that does not collide with on-premises or other Azure networks. **Why required:** VNet peering fails or routes incorrectly when address spaces overlap; this cannot be fixed after workloads land without re-addressing. **How to verify:** Confirm the chosen hub/spoke CIDRs against your IPAM / existing network inventory; the discovery script lists existing VNet ranges. **How to implement if missing:** Allocate the ranges from your IPAM before deploying the network module; encode them as parameters in the Bicep, never as hard-coded literals.

R6. Naming and tagging convention encoded as parameters

Description: The organization's naming convention and required tags (per Global Requirements G7) expressed as Bicep parameters/variables so every resource is named and tagged consistently. **Why required:** Policy will enforce naming/tags; inconsistent names break governance, cost allocation, and the deny guardrails. **How to verify:** The repo has a `parameters/` file (or `.bicepparam`) defining prefixes, environment codes, and the required tag set. **How to implement if missing:** Capture the convention as parameters before first deploy; the Implementation Guide names every object with the `<Purpose> – <Scope> – <Stage>` convention.

R7. Pipeline approval gate and reviewer group

Description: A named group of approvers and a configured **environment approval** (Azure DevOps *Environment* checks, or a GitHub *Environment* with required reviewers) that the production stage must pass. **Why required:** No production apply runs without human approval after the `what-if` review; this is the control that makes IaC safe. **How to verify:** The `Prod` environment in the pipeline platform lists required reviewers; a test run pauses at the gate. **How to implement if missing:** Create the environment, add the reviewer group, and reference it from the production stage (shown in `azure-pipelines.yml`). *Who may approve production changes is the customer's change-control decision.*

Global Requirements — DevOps (category-level)

These requirements apply to every solution in the DevOps category (Infrastructure-as-Code, pipelines, policy-as-code). They are published once at the category level so individual listings stay small. Confirm these before deploying any IaC/automation solution. Each entry: Description / Why / Verify / Implement-if-missing.

Infrastructure-as-code changes real cloud resources. Everything runs plan/what-if first, through a reviewed pipeline, against a non-production target before production — never an unreviewed apply against prod. State and pipeline credentials are treated as secrets.

G1. Target cloud subscription/tenant with sufficient rights

Description: An Azure subscription (and Entra tenant) where resources will be created, with Owner or an equivalent scoped role at the deployment scope. Why: Landing-zone and policy deployments create management groups, role assignments, and resources that need elevated rights. Verify: `az account show` succeeds; the identity holds Owner/User Access Administrator at the target scope. Implement: Obtain a subscription and the required role assignment (or a service principal / workload identity with those rights).

G2. IaC tooling installed and versioned

Description: The required toolchain — Terraform CLI, Bicep CLI / Azure CLI, and Git — at pinned versions. Why: IaC is version-sensitive; provider/module/CLI drift breaks reproducibility. Verify: `terraform version`, `az bicep version`, `az version`, and `git --version` return supported builds. *Verify current provider/module versions.* Implement: Install and pin the toolchain versions; record them in the repo.

G3. Source control repository

Description: A Git repository (Azure Repos or GitHub) to hold the IaC and pipeline definitions. Why: Code review, history, and CI/CD all depend on version control; IaC not in Git defeats the purpose. Verify: The repo exists and the operator can clone/push. Implement: Create the repository and set branch protection before authoring infrastructure code.

G4. CI/CD platform and service connection

Description: A pipeline platform (Azure DevOps Pipelines or GitHub Actions) with a secure connection to the cloud (workload identity federation preferred over secrets). Why: Plan/apply must run through an auditable pipeline with least-privilege, not from a laptop. Verify: A pipeline can authenticate to the subscription via OIDC/workload identity or a scoped service principal. Implement: Configure the service connection using workload identity federation; scope its rights to the deployment target.

G5. Remote state / deployment backend

Description: A secure remote backend for state (e.g. Azure Storage for Terraform state) or Azure deployment stacks for Bicep. Why: Local state is unsafe and unshareable; remote state with locking prevents corruption and drift. Verify: The state storage account/container (or deployment-stack scope) exists and is access-controlled. Implement: Provision the state backend with encryption, versioning, and least-privilege access before first apply.

G6. Secrets management

Description: A secrets store (Azure Key Vault) for any credentials the pipeline needs; no secrets in code. Why: Committed secrets leak; IaC repos are frequently shared and reviewed. Verify: No secrets in the

repo (scanning enabled); pipeline pulls secrets from Key Vault / secure variables. Implement: Move all secrets to Key Vault / pipeline secret store; enable secret scanning on the repo.

G7. Naming, tagging, and region standards

Description: An organizational naming convention, required tags, and allowed regions. Why: Consistent naming/tagging is needed for governance, cost management, and policy enforcement. Verify: A documented standard exists and is encoded in the IaC modules/policy. Implement: Define the standard and encode it as policy-as-code and module defaults.

G8. Change-control and non-production target

Description: A non-production subscription/scope to validate against, and a change-approval process for production. Why: Every change is validated with plan/what-if against non-prod and approved before prod apply. Verify: A non-prod target exists; the pipeline requires approval before prod stages. Implement: Provision a non-prod scope and add manual approval gates to the production pipeline stages. *Production change approval is the customer's operational decision.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — Azure Landing Zone with Bicep

Follow these steps in order. Everything is **pilot-first**: you deploy against a **non-production** management group with policy in **audit**, validate, then expand to production behind pipeline approval and promote guardrails to **deny**. Every object you create is named with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in later steps, testing, and the scripts. Placeholder tenant is `contoso.com` / `contoso.onmicrosoft.com`.

> *Run every apply with* `what-if` ***first****. Never run an unreviewed apply against production. All examples use the Azure CLI; the pipeline runs the same commands.*

Naming you will create (reuse these exact names)

Object	Name
Intermediate root management group	`alz` (display: `Landing Zone – Tenant – Root`)
Platform management group	`Platform – Tenant – Prod`
Landing-zones management group	`Landing Zones – Tenant – Prod`
Sandbox / non-prod management group	`Sandbox – Tenant – NonProd`
Guardrail initiative assignment	`Guardrails Baseline – Landing Zones – Audit` (later `…– Deny`)
Custom RBAC role	`Landing Zone Contributor – Platform – Baseline`
Hub virtual network	`Hub Network – Connectivity – Prod`
First spoke virtual network	`Spoke Network – Workload01 – NonProd`

Object	Name
Deployment stack (non-prod)	`alz-stack – Sandbox – NonProd`
Deployment stack (prod)	`alz-stack – Platform – Prod`
Pipeline	`Landing Zone Pipeline – Platform – CICD`

Step 1 — Assess: inventory the current estate (read-only)

1. Sign in: `az login` (add `--tenant contoso.onmicrosoft.com` if you have several).
2. Set the working subscription: `az account set --subscription "<non-prod sub id>"`.
3. Run the discovery script from the `scripts/` folder:

`powershell -ExecutionPolicy Bypass -File .\IACBICEP-Discovery.ps1` It reports existing management groups, policy assignments, custom roles, standing **Owner** assignments, and existing VNet address ranges. Save the output — it is your baseline and your address-overlap check for **R5**.

1. Confirm your identity has the rights from **R1** at the intermediate group.

Step 2 — Configure: scaffold the Bicep repo

1. Clone the empty repo created in Global Requirements G3 and add the provided files at the root:

• `main.bicep` — the landing-zone template (entry point).

• `azure-pipelines.yml` — the CI/CD pipeline.

• `IACBICEP-Discovery.ps1`, `IACBICEP-Validate.ps1`, `IACBICEP-Report.ps1` under `scripts/`.

1. Create a `bicepconfig.json` and pin the Bicep version (**R2**), and a parameters file (`main.nonprod.bicepparam` and `main.prod.bicepparam`) holding your naming prefixes, tag set, address ranges (**R5/R6**), and the target management-group IDs.
2. Commit to a **feature branch** and open a pull request — infrastructure changes are reviewed, never pushed straight to `main` (Global Requirements G3 branch protection).

Step 3 — Configure: deploy the management-group hierarchy (non-prod first)

1. Preview with what-if at tenant/management-group scope:

```
az deployment mg what-if --management-group-id alz --location eastus
--template-file main.bicep --parameters main.nonprod.bicepparam --parameters
deployHierarchy=true
```

1. Read the preview. It should show the creation of `Platform – Tenant – Prod`, `Landing Zones – Tenant – Prod`, and `Sandbox – Tenant – NonProd` under `alz`.
2. Deploy the hierarchy as a deployment stack (non-prod scope):

```
az stack mg create --name "alz-stack – Sandbox – NonProd"
--management-group-id alz --location eastus --template-file main.bicep
--parameters main.nonprod.bicepparam --parameters deployHierarchy=true
--deny-settings-mode denyDelete --action-on-unmanage detachAll
```

1. Verify in the portal: **Management groups** now shows the tree.

Step 4 — Configure: assign policy guardrails in AUDIT

1. What-if the policy assignment scoped to `Landing Zones – Tenant – Prod`:

```
az deployment mg what-if --management-group-id alz --location eastus
--template-file main.bicep --parameters main.nonprod.bicepparam --parameters
deployPolicy=true policyEffect=Audit
```

1. Deploy it (updates the same stack). This creates the initiative assignment `Guardrails Baseline – Landing Zones – Audit` — require encryption, block public storage, allowed locations, allowed SKUs, require tags — all in **audit** effect.

2. Confirm under **Policy** → **Assignments** at the management-group scope that the assignment exists and its effect is **Audit**. Do **not** move to deny yet.

Step 5 — Configure: apply the identity / RBAC baseline

1. What-if the RBAC module, then deploy it. It creates the custom role `Landing Zone Contributor – Platform – Baseline` (narrow actions, no `Microsoft.Authorization/*/write`) and assigns it at `Platform – Tenant – Prod`.

2. Review existing standing **Owner** assignments from the Step 1 discovery output. Plan their removal (do this in a controlled change — removing your own Owner mid-deploy can lock you out).

3. Where **Entra ID P2** is licensed, configure the eligible (PIM) assignments the Bicep references. *Verify current PIM availability and licensing in your tenant.*

Step 6 — Configure: deploy hub-spoke networking and DNS (non-prod)

1. What-if the network module with your **R5** address ranges:

```
az deployment sub what-if --subscription "<non-prod sub id>" --location eastus
--template-file main.bicep --parameters main.nonprod.bicepparam --parameters
deployNetwork=true
```

1. Deploy it. This creates `Hub Network – Connectivity – Prod` (with `GatewaySubnet`, `AzureFirewallSubnet`, and a shared-services subnet), the first spoke `Spoke Network – Workload01 – NonProd`, the hub↔spoke **peerings**, and the **private DNS zones** (e.g. `privatelink.blob.core.windows.net`) linked to the hub.

2. Confirm peering status is **Connected** in the portal (**Virtual networks** → **Peerings**).

Step 7 — Pilot: validate the non-prod landing zone

1. Run the validation script (what-if / read-only):

`powershell -ExecutionPolicy Bypass -File .\IACBICEP-Validate.ps1 -ManagementGroupId alz -Environment NonProd` It confirms the hierarchy exists, the guardrail assignment is present in **Audit**, the custom role is assigned, and peerings are Connected — see the **Testing** guide for pass/fail gates.

1. Run a policy compliance scan and review audit findings (`az policy state trigger-scan`). Exempt any legitimate false positives before you ever consider deny.
2. Generate the posture report: `powershell -ExecutionPolicy Bypass -File .\IACBICEP-Report.ps1 -ManagementGroupId alz`.

Step 8 — Wire the CI/CD pipeline (what-if → approval → deploy)

1. In Azure DevOps (or GitHub), create the pipeline `Landing Zone Pipeline – Platform – CICD` from `azure-pipelines.yml`.
2. Confirm the **service connection** uses workload identity federation (Global Requirements G4) and the `Prod` **environment** has the approval gate and reviewer group from **R7**.
3. The pipeline stages are: **Validate** (lint/build Bicep) → **What-If** (posts the preview to the PR) → **Approval** (manual gate) → **Deploy** (deployment stack). Merging to `main` triggers Validate + What-If; the Deploy stage only runs after approval.

Step 9 — Expand: deploy to production, then promote guardrails to DENY

1. With `main.prod.bicepparam`, let the pipeline run **Validate** → **What-If** against production. Review the what-if output in the PR.
2. Approve the gate. The pipeline deploys the production stack `alz-stack – Platform – Prod`.
3. Only after production has run cleanly in **audit** for your agreed soak period, change the policy effect to deny: re-run with `policyEffect=Deny`, review the what-if, and approve. The assignment becomes `Guardrails Baseline – Landing Zones – Deny`.
4. **Rollback** at any point: `az stack mg delete/update` on the named stack removes or reverts exactly what it deployed (use `--action-on-unmanage detachAll` to leave resources in place while detaching stack management).

Onboard each new subscription by moving it under `Landing Zones – Tenant – Prod` and adding a spoke to the parameters — it inherits policy, RBAC, and connectivity automatically. That is the repeatable payoff of the landing zone.

5. Testing — Azure Landing Zone with Bicep

Validate the landing zone against a **non-production** management group before you expand to production. Each test has an explicit **pass/fail gate**. Reference the scripts by name; they are read-only / what-if except where noted.

Test 1 — Management-group hierarchy exists

• **How:** `powershell -ExecutionPolicy Bypass -File .\IACBICEP-Validate.ps1 -ManagementGroupId alz -Environment NonProd`, or in the portal open **Management groups**.

• **Pass:** `Platform – Tenant – Prod`, `Landing Zones – Tenant – Prod`, and `Sandbox – Tenant – NonProd` all appear under `alz`.

• **Fail:** Any group missing → re-run Step 3's what-if and check the `deployHierarchy` parameter and your R1 rights.

Test 2 — Bicep builds and what-if is clean

• **How:** `az bicep build --file main.bicep` then `az deployment mg what-if ... --parameters deployHierarchy=true deployPolicy=true`.

• **Pass:** Build produces no errors; what-if shows **no unexpected deletes** and only the changes you intend.

• **Fail:** Build errors → check the pinned Bicep version (R2). Unexpected deletes in what-if → **stop**; do not deploy until you understand them.

Test 3 — Guardrails assigned in AUDIT (not deny yet)

• **How:** `IACBICEP-Validate.ps1` reports every policy assignment and its effect; or **Policy** → **Assignments** at the management-group scope.

• **Pass:** `Guardrails Baseline – Landing Zones – Audit` exists with effect **Audit**.

• **Fail:** Effect already **Deny** during pilot → revert to audit (you have promoted too early). Assignment missing → re-run Step 4.

Test 4 — Audit compliance picture is understood

• **How:** `az policy state trigger-scan --management-group alz`, then `IACBICEP-Report.ps1 -ManagementGroupId alz`.

• **Pass:** The report lists non-compliant resources; every one is either a real finding to fix or an intended exemption. No surprises.

• **Fail:** Compliance state still `NotStarted` → wait and re-scan (evaluation is not instantaneous). Unexpected non-compliance → resolve or exempt **before** considering deny.

Test 5 — Least-privilege RBAC baseline

• **How:** `IACBICEP-Validate.ps1` reports the custom role and standing Owner count; or **Access control (IAM)** → **Role assignments**.

• **Pass:** `Landing Zone Contributor – Platform – Baseline` is assigned at `Platform – Tenant – Prod`; standing Owner assignments are only the intended break-glass account(s).

• **Fail:** Extra standing Owners → plan their controlled removal (Step 5). Custom role missing → re-run the RBAC module.

Test 6 — Hub-spoke connectivity and DNS

• **How:** `IACBICEP-Validate.ps1` checks peering state and DNS-zone links; or **Virtual networks** → **Peerings**.

• **Pass:** `Hub Network – Connectivity – Prod` and `Spoke Network – Workload01 – NonProd` exist; peerings are **Connected** both directions; the private DNS zones are linked to the hub.

• **Fail:** Peering `Disconnected` → address-space overlap or a one-sided peering (R5); re-check ranges and re-deploy the network module.

Test 7 — Deployment stack tracks and can roll back

• **How:** `az stack mg show --name "alz-stack – Sandbox – NonProd" --management-group-id alz`.

• **Pass:** The stack lists its managed resources; a `what-if` of an `az stack ... delete/update` shows exactly what would be removed/reverted.

• **Fail:** Stack not found → you deployed with plain `az deployment` instead of `az stack`; redeploy via the stack so rollback is available.

Test 8 — Pipeline enforces what-if and approval

• **How:** Push a trivial parameter change on a branch; open a PR; watch the pipeline.

• **Pass: Validate** and **What-If** run automatically and post the preview; the **Deploy** stage **pauses at the approval gate** (R7) and only proceeds after an approver signs off.

• **Fail:** Deploy runs without pausing → the environment approval check is not attached to the production stage; fix in the pipeline environment configuration.

Exit criteria

All eight tests pass against the non-production management group, the audit compliance picture is clean or fully exempted, and the pipeline pauses for approval — **only then** expand to production and (after the soak period) promote guardrails to deny.

6. Troubleshooting — Azure Landing Zone with Bicep

Common failures during a Bicep landing-zone rollout, as **symptom** → **cause** → **fix**. Re-run `IACBICEP-Validate.ps1` after any fix to re-check the gates.

Deployment authorization / scope

Symptom: `AuthorizationFailed` or `does not have permission to perform action 'Microsoft.Management/managementGroups/write'` on the hierarchy deploy. **Cause:** The identity lacks Owner (or Contributor + User Access Administrator) at the intermediate management group (**R1**). **Fix:** Have a Global Administrator grant the role at `alz`; confirm with `az account management-group list` and re-run Step 3's what-if.

Symptom: `MissingSubscriptionRegistration` for `Microsoft.Network` / `Microsoft.Authorization`. **Cause:** The resource provider is not registered on the target subscription

(**R4**). **Fix:** `az provider register --namespace Microsoft.Network`; registration is async — wait for `Registered`, then redeploy.

Bicep build / what-if

Symptom: `az bicep build` fails or `az stack` is "not a recognized command". **Cause:** Bicep / Azure CLI too old, or the version is not pinned (**R2/R3**). **Fix:** `az bicep upgrade` and update the CLI; pin versions in `bicepconfig.json`. *Verify current supported versions.*

Symptom: `what-if` shows unexpected **Delete** or **Modify** on resources you did not intend to change. **Cause:** A parameter changed a name (so ARM sees a replace), or someone edited a stack-managed resource in the portal (drift). **Fix: Stop — do not approve.** Diff the parameter files, run `IACBICEP-Report.ps1` for drift, reconcile, then re-what-if until the preview is clean.

Deployment stacks / rollback

Symptom: Stack update fails with `denyAssignment` blocking a change. **Cause:** The stack's `denyDelete`/deny-settings are protecting managed resources from out-of-band change — working as intended. **Fix:** Make the change **through the stack** (update the Bicep + redeploy the stack), not directly on the resource.

Symptom: `az stack mg show` reports the stack is missing after you deployed. **Cause:** You deployed with `az deployment mg create` instead of `az stack mg create`, so there is no stack (and no rollback unit) — Test 7 fail. **Fix:** Redeploy via `az stack mg create` with the named stack so the resources are tracked and rollback is available.

Policy

Symptom: A brand-new guardrail shows every resource as **NonCompliant** immediately, alarming stakeholders. **Cause:** The assignment is in **Audit** (correct) but the compliance scan surfaced the true current state — this is the point of audit-before-deny. **Fix:** Triage findings, fix real issues, and **exempt** legitimate cases before promoting to deny. Never move to deny with an un-triaged audit picture.

Symptom: Compliance state stays `NotStarted` / `Unknown` after assignment. **Cause:** Policy evaluation is not instantaneous. **Fix:** `az policy state trigger-scan --management-group alz`, wait, then re-run `IACBICEP-Report.ps1`.

Symptom: A legitimate production deployment starts failing after you promoted guardrails to **Deny**. **Cause:** A deny policy is now blocking a real, allowed pattern that was only visible as an audit finding before. **Fix:** Revert the effect to Audit (re-run with `policyEffect=Audit`), fix or exempt the pattern, then re-promote. This is why the model soaks in audit first.

RBAC / identity

Symptom: You lose access mid-deployment after the RBAC step. **Cause:** Removing standing **Owner** included your own account. **Fix:** Restore access via the documented break-glass account; always exclude break-glass from Owner-removal and stage RBAC changes carefully (Step 5).

Symptom: PIM-eligible assignment in the Bicep fails to create. **Cause: Entra ID P2** not licensed, or PIM not enabled in the tenant. **Fix:** *Verify PIM availability and licensing in your tenant.* Fall back to a scoped

standing assignment of the least-privilege custom role until P2 is in place.

Networking

Symptom: Hub↔spoke peering shows **Disconnected** / one-sided. **Cause:** Overlapping address spaces or only one direction of the peering was created (**R5**). **Fix:** Confirm non-overlapping CIDRs from the discovery output, ensure both peering directions exist, and redeploy the network module.

Symptom: Private-endpoint DNS does not resolve from a spoke. **Cause:** The private DNS zone is linked to the hub but not reachable from the spoke, or the zone name is wrong. **Fix:** Verify the private DNS zone name (e.g. `privatelink.blob.core.windows.net`), confirm the VNet link and that spoke traffic routes through the hub, then re-test resolution.

Pipeline

Symptom: The Deploy stage runs without pausing for approval. **Cause:** The Environment approval check (required reviewers) is not attached to the production stage (**R7**). **Fix:** Add required reviewers to the `landing-zone-prod` Environment and confirm the deploy job references `environment:` — re-run the pipeline (Test 8).

Symptom: Pipeline auth fails with a token/credential error. **Cause:** The service connection is not configured for workload identity federation, or the federated credential subject/scope is wrong (**G4**). **Fix:** Reconfigure the service connection for OIDC/workload identity federation and scope it to the deployment target; never fall back to a stored secret.

7. Best Practices — Azure Landing Zone with Bicep

Keep it clean

• **Everything through the pipeline.** After the initial bootstrap, never `az deployment`/`az stack` from a laptop against production. The pipeline's what-if → approval → deploy path is the only production route, so every change is reviewed and recorded.

• **What-if is a gate, not a formality.** Read every what-if before approving. An unexpected **delete** or **modify** on a resource you did not intend to touch means **stop** — investigate before you approve.

• **Audit before deny, always.** Assign new guardrails in audit, understand the compliance picture, exempt legitimate cases, then promote to deny. Never ship a new policy straight to deny in production.

• **Pin your toolchain.** Pin the Bicep and Azure CLI versions in the repo (`bicepconfig.json` / a tool-version file). IaC that "worked last month" breaks when a CLI silently updates. *Verify current supported versions periodically.*

• **Parameterize, never hard-code.** Address ranges, tags, prefixes, and management-group IDs live in `.bicepparam` files — one file per environment. The same `main.bicep` deploys non-prod and prod; only parameters differ.

• **Use deployment stacks for rollback.** Deploy as stacks with `denyDelete` so out-of-band portal edits are blocked, and keep `--action-on-unmanage` explicit so a rollback does exactly what you expect.

• **Least privilege by default.** No standing Owner except a documented break-glass account; prefer custom roles scoped to the narrowest management group; use PIM-eligible assignments where Entra ID P2 is licensed (*verify licensing*).

• **Modularize.** Keep hierarchy, policy, RBAC, and network as separate Bicep modules toggled by parameters (`deployHierarchy`, `deployPolicy`, `deployNetwork`) so you can what-if and deploy each independently.

• **Secrets stay out of code.** Workload-identity federation for the pipeline (no secret), Key Vault for anything a resource needs, and secret scanning on the repo (Global Requirements G4/G6).

• **Non-overlapping addressing forever.** Reserve hub/spoke ranges in IPAM up front; re-addressing after workloads land is painful. The discovery script's VNet inventory is your overlap check.

Governance and decisions that belong to the customer

• **Production change approval**, who may approve, and the soak period before deny are the customer's **change-control** decisions — the solution provides the technical gate, not the policy.

• **Regulatory / data-residency** determinations (allowed regions, sovereign-cloud requirements) belong to the customer's compliance function; the guardrails **enforce** those choices once made.

• **Break-glass account** ownership and testing cadence are the customer's security decision.

Where to go next

Once the landing zone is live, adjacent catalog areas extend it:

• **Azure Solutions** — *Azure Security Baseline & Governance* deepens the policy/Defender-for-Cloud posture on top of the hierarchy you just built.

• **DevOps** — the sibling *Azure Landing Zone with Terraform* solution if a team standardizes on Terraform instead of Bicep, and *Intune-as-Code (Graph & Terraform)* to bring endpoint management under the same code-and-pipeline discipline.

• **Security / Identity** — Conditional Access and PIM solutions to harden the identity baseline this landing zone starts.

• **Networking** — extend hub-spoke with Azure Firewall policy, VPN/ExpressRoute gateways, and private-endpoint patterns per workload.

8. Deliverables Summary — Azure Landing Zone with Bicep

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What an Azure landing zone is — management groups, Azure Policy as Bicep, RBAC baseline, deployment stacks, hub-spoke networking/DNS; how Bicep and what-if work; benefits and limitations

Document	Purpose
2. Requirements	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (DevOps)	Category-wide prerequisites, published once
3. Implementation	Exact, ordered steps — scaffold repo, deploy hierarchy, assign policy in audit, RBAC baseline, hub-spoke network, CI/CD, then prod + deny
5. Testing	Validation procedures with pass/fail gates
6. Troubleshooting	Field issues → cause → fix
7. Best Practices	Field-proven recommendations and where to go next

IaC and automation artifacts

File	What it does
`IACBICEP-Discovery.ps1`	Read-only estate discovery — existing management groups, policy assignments, custom roles, standing Owner assignments, and VNet address ranges (the address-overlap check)
`main.bicep`	The core landing-zone template — management-group hierarchy, policy/initiative assignments, custom-role + RBAC baseline, and hub-spoke network with private DNS, toggled by `deployHierarchy` / `deployPolicy` / `deployNetwork` parameters
`IACBICEP-Validate.ps1`	What-if / read-only validation — confirms the hierarchy, guardrail assignment (in Audit), custom-role assignment, and hub-spoke peering/DNS against the pass/fail gates
`azure-pipelines.yml`	CI/CD pipeline (Azure DevOps; GitHub Actions notes inline) — Validate → What-If → **approval gate** → Deploy (deployment stack), what-if-first and approval-gated
`IACBICEP-Report.ps1`	Governance posture report — policy compliance, non-compliant resources, RBAC/standing-Owner summary, and drift against the deployed stack

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five IaC / automation artifacts (`main.bicep`, `azure-pipelines.yml`, and three PowerShell scripts) in a single ZIP.

- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — what-if-first, approval-gated, with deployment-stack rollback.

Delivered as a Complete Solution Package (PDF + Word + scripts).

Outcome

A Microsoft-native, versioned Azure foundation expressed entirely as Bicep in Git — management groups, policy guardrails, least-privilege identity, and hub-spoke connectivity — governed and repeatable, proven on a non-production scope before it reaches production.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IACBICEP-Discovery.ps1

Purpose: Discovery for the "Azure Landing Zone with Bicep" solution. Reports the existing management-group hierarchy, policy assignments and their effect, custom role definitions, standing Owner role assignments, and every virtual-network address range (the address-overlap check for the hub-spoke plan). Makes NO changes — this is your baseline snapshot. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IACBICEP-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\IACBICEP-Discovery.ps1 -ManagementGroupId alz Requires the Az modules (Az.Accounts, Az.Resources, Az.Network). Run Connect-AzAccount first, or the script will prompt. *Verify current Az module versions — cmdlet surfaces change between versions.* EXPECTED RESULTS A per-section snapshot: management groups, policy assignments (with effect), custom roles, standing Owner assignments (highlighted), and existing VNet CIDRs — a single readiness picture before you scaffold main.bicep and deploy. .NOTES White-label. Read-only — reports on configuration/identifiers only; makes no changes. > [CmdletBinding()] param([string]$ManagementGroupId = 'alz', [string]$SubscriptionId)

How to run: powershell -ExecutionPolicy Bypass -File .\IACBICEP-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\IACBICEP-Discovery.ps1 -ManagementGroupId alz Requires the Az modules (Az.Accounts, Az.Resources, Az.Network). Run Connect-AzAccount first, or the script will prompt. *Verify current Az module versions — cmdlet surfaces change between versions.* EXPECTED RESULTS A per-section snapshot: management groups, policy assignments (with effect), custom roles, standing Owner assignments (highlighted), and existing VNet CIDRs — a single readiness picture before you scaffold main.bicep and deploy. .NOTES White-label. Read-only — reports on configuration/identifiers only; makes no changes. > [CmdletBinding()] param([string]$ManagementGroupId = 'alz', [string]$SubscriptionId)

IACBICEP-Report.ps1

Purpose: Produces a read-only posture report for the deployed landing zone: policy-compliance summary and the list of non-compliant resources, the guardrail assignment and its effect, an RBAC / standing-Owner summary, and deployment-stack drift (resources the stack manages vs. what exists). Useful before promoting guardrails from Audit to Deny and on each governance cycle. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IACBICEP-Report.ps1 -ManagementGroupId alz powershell -ExecutionPolicy Bypass -File .\IACBICEP-Report.ps1 -ManagementGroupId alz -OutFile posture.csv Requires Az modules (Az.Accounts, Az.Resources, Az.PolicyInsights). Run Connect-AzAccount first. Verify current Az module versions.* EXPECTED RESULTS A console report (and optional CSV) of compliance, guardrail effect, RBAC posture, and drift. No changes are made. .NOTES White-label. Read-only reporting. Makes no changes. > [CmdletBinding()] param([string]$ManagementGroupId = 'alz', [string]$StackName = 'alz-stack – Sandbox – NonProd', [string]$OutFile)

How to run: powershell -ExecutionPolicy Bypass -File .\IACBICEP-Report.ps1 -ManagementGroupId alz powershell -ExecutionPolicy Bypass -File .\IACBICEP-Report.ps1 -ManagementGroupId alz -OutFile posture.csv Requires Az modules (Az.Accounts, Az.Resources, Az.PolicyInsights). Run Connect-AzAccount first. Verify current Az module versions.* EXPECTED RESULTS A console report (and optional CSV) of compliance, guardrail effect, RBAC posture, and drift. No changes are made. .NOTES White-label. Read-only reporting. Makes no changes. > [CmdletBinding()] param(

[string]$ManagementGroupId = 'alz', [string]$StackName = 'alz-stack – Sandbox – NonProd', [string]$OutFile)

IACBICEP-Validate.ps1

Purpose: Confirms the landing zone matches the Testing guide's pass/fail gates: the management-group hierarchy exists, the guardrail policy is assigned in the EXPECTED effect (Audit during pilot), the least-privilege custom-role assignment is present, standing Owner is minimal, and hub-spoke peerings are Connected. Runs `bicep build` and a management-group `what-if` so you can see any pending change WITHOUT deploying. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IACBICEP-Validate.ps1 -ManagementGroupId alz -Environment NonProd Requires the Azure CLI (az, az bicep) and the Az PowerShell modules. Run az login / Connect-AzAccount first. Verify current az / Az module versions.* EXPECTED RESULTS A PASS/FAIL line per gate and the what-if preview text. No changes are made. .NOTES White-label. Read-only / what-if only. Does not deploy. > [CmdletBinding()] param([string]$ManagementGroupId = 'alz', [ValidateSet('NonProd','Prod')] [string]$Environment = 'NonProd', [string]$TemplateFile = '..\main.bicep', [string]$ExpectedPolicyEffect = 'Audit')

How to run: powershell -ExecutionPolicy Bypass -File .\IACBICEP-Validate.ps1 -ManagementGroupId alz -Environment NonProd Requires the Azure CLI (az, az bicep) and the Az PowerShell modules. Run az login / Connect-AzAccount first. Verify current az / Az module versions.* EXPECTED RESULTS A PASS/FAIL line per gate and the what-if preview text. No changes are made. .NOTES White-label. Read-only / what-if only. Does not deploy. > [CmdletBinding()] param([string]$ManagementGroupId = 'alz', [ValidateSet('NonProd','Prod')] [string]$Environment = 'NonProd', [string]$TemplateFile = '..\main.bicep', [string]$ExpectedPolicyEffect = 'Audit')

azure-pipelines.yml

Purpose: What-if-first, approval-gated delivery of main.bicep. Stages: Validate -> lint + build the Bicep WhatIf -> management-group what-if, published as the change preview Deploy -> deployment-stack apply, BEHIND a manual approval on the 'landing-zone-prod' Environment (required reviewers = R7) HOW TO RUN Create the pipeline "Landing Zone Pipeline – Platform – CICD" from this file. Set the service connection (workload identity federation — no secret, G4) in the variables below, and add required reviewers to the 'landing-zone-prod' Environment so the Deploy stage pauses for approval. EXPECTED RESULTS PRs run Validate + WhatIf automatically; the preview is attached. Deploy only runs after a human approves the Environment check. Nothing applies unreviewed. White-label. Plan/what-if first, approval-gated. GitHub Actions equivalent noted at the bottom. *Verify current task versions and az CLI availability on the agent.*
==========

How to run: Create the pipeline "Landing Zone Pipeline – Platform – CICD" from this file. Set the service connection (workload identity federation — no secret, G4) in the variables below, and add required reviewers to the 'landing-zone-prod' Environment so the Deploy stage pauses for approval. EXPECTED RESULTS PRs run Validate + WhatIf automatically; the preview is attached. Deploy only runs after a human approves the Environment check. Nothing applies unreviewed. White-label. Plan/what-if first, approval-gated. GitHub Actions equivalent noted at the bottom. *Verify current task versions and az CLI availability on the agent.*
==========

main.bicep

Purpose: // Declares the landing-zone foundation as code: a management-group hierarchy, // an Azure Policy guardrail initiative assignment (audit -> deny via param), // a least-privilege custom role assignment, and a hub-spoke network with a // private DNS zone. Each block is toggled by a parameter so you can what-if // and deploy modules independently. // // HOW TO RUN (what-if FIRST, always) // az deployment mg what-if \ // --management-group-id alz --location eastus \ // --template-file main.bicep --parameters main.nonprod.bicepparam \ // --parameters deployHierarchy=true deployPolicy=true // Deploy as a deployment stack (rollback-capable): // az stack mg create --name "alz-stack – Sandbox – NonProd" \ // --management-group-id alz --location eastus \ // --template-file main.bicep --parameters main.nonprod.bicepparam \ // --deny-settings-mode denyDelete --action-on-unmanage detachAll // // EXPECTED RESULTS // The management-group tree, the guardrail assignment in the requested effect, // the custom-role assignment, and the hub-spoke VNets + private DNS — created // idempotently. Re-running converges; it does not duplicate. // // White-label. Deploy plan/what-if first, through the reviewed pipeline. // NOTE: API versions below are marked *verify* — confirm current supported // apiVersion values and deployment-stacks availability in your tenant/cloud. //
==========